

Recipe Shelter — Règles métier (Backend)

Table of Contents

Recipe Shelter — Règles métier (Backend)

Document généré à partir de l'analyse du code source du backend (`recipe-shelter-backend-main`) : services, DTO/validateurs, middlewares, schéma SQL et données d'amorçage (seed). Il recense les règles métier telles qu'implémentées, indépendamment de la documentation technique d'installation (README).

Sommaire

1. [Comptes & authentification](#)
 2. [Autorisations \(RBAC\)](#)
 3. [Recettes](#)
 4. [Catalogue \(ingrédients, tags, catégories, ustensiles\)](#)
 5. [Propositions de catalogue \(contributions communautaires\)](#)
 6. [Commentaires & notation](#)
 7. [Favoris](#)
 8. [Modération & journal d'audit administratif](#)
 9. [Gestion du personnel \(Staff\)](#)
 10. [Formulaire de contact](#)
 11. [Images de recettes](#)
 12. [Sécurité transverse](#)
 13. [Annexe — invariants au niveau base de données](#)
-

1. Comptes & authentification

1.1 Deux populations de comptes étanches

Un compte est soit **community** (utilisateur du site), soit **staff** (personnel d'administration). Ce n'est jamais les deux, et les règles ci-dessous s'appliquent séparément :

- Deux royaumes de session distincts (`app` pour `community`, `admin` pour `staff`), avec cookies, audiences JWT et durées de vie différentes (voir [12.2](#)).
- Un statut de compte `staff` ne peut jamais s'authentifier sur le royaume `app`, et inversement.
- Un compte `staff` est nécessairement enrôlé en double-facteur WebAuthn : la base de données interdit explicitement un `staff` `active` sans `MfaEnrolledAt`.

1.2 Inscription communautaire (POST /auth/register)

- Champs requis : email, nom d'utilisateur, mot de passe.
- Email normalisé (minuscules, espaces retirés) et doit contenir @.
- Nom d'utilisateur : minimum **3 caractères** après suppression des espaces.
- Mot de passe : voir [politique de mot de passe](#).
- Email et nom d'utilisateur doivent être uniques (sinon conflit 409).
- Le compte est créé avec le statut **inactive** — il ne peut pas se connecter tant que l'email n'est pas validé.
- Un email de validation est envoyé automatiquement à la création.

1.3 Validation d'email

- Le lien de validation est un token à usage unique, valable **24 heures** (1440 minutes).
- Un renvoi de l'email de validation (`resend-validation-email`) n'est autorisé que pour un compte `community` au statut `inactive` ; toute autre situation est refusée. Si l'email n'existe pas, la demande est silencieusement ignorée (pas de fuite d'information sur l'existence d'un compte).
- Toute nouvelle demande de renvoi invalide les tokens de validation précédemment émis pour cet utilisateur.
- La validation échoue si : le token est invalide, déjà utilisé, expiré, si le compte n'est pas `community`, ou si le compte est banni.
- Succès — le compte passe au statut **active**.

1.4 Connexion communautaire (POST /auth/login)

- Vérifie email + mot de passe (`bcrypt`).
- Refusée si : identifiants invalides, compte non `community`, compte `inactive` (email non validé), compte banni.
- Succès — émission d'un token de session `app` (cookie `HttpOnly` dédié).

1.5 Politique de mot de passe

Appliquée à l'inscription, à la réinitialisation, au changement de mot de passe et à l'activation d'un compte `staff` :

- Longueur minimale : **8 caractères**.
- Longueur maximale : **128 caractères**.

1.6 Réinitialisation de mot de passe

- Demande (`forgot-password`) : toujours une réponse neutre, que l'email existe ou non (pas de fuite d'information). Un token est généré, valable **30 minutes**, et toute demande antérieure non utilisée est invalidée.
- Réinitialisation (`reset-password`) : le nouveau mot de passe doit respecter la politique ci-dessus ; le token doit être valide et non expiré. Un email de confirmation de changement est envoyé après succès.

1.7 Mise à jour du profil (compte connecté)

Pour changer l'email, le mot de passe ou le nom d'utilisateur (`/users/me/...`), le mot de passe **actuel** doit systématiquement être revérifié :

Action	Règles
Changer l'email	Nouvel email requis, format valide, différent de l'actuel, non déjà utilisé par un autre compte.
Changer le mot de passe	Le nouveau mot de passe doit être différent de l'actuel et respecter la politique de mot de passe.
Changer le nom d'utilisateur	Minimum 3 caractères, différent de l'actuel, non déjà pris.

1.8 Comptes staff & double authentication WebAuthn

- Un compte staff est créé via **invitation** (voir 9.1) et ne peut se connecter qu'après avoir complété son **enrôlement MFA WebAuthn** (obligatoire, non désactivable).
- La connexion staff est un flux en deux temps :
 1. `login/options` : vérifie email + mot de passe, exige un compte staff avec un `sessionVersion` valide et strictement positif, puis initie un challenge WebAuthn (résident, vérification utilisateur obligatoire).
 2. `login/verify` : valide l'assertion WebAuthn contre le challenge (expirant sous 10 minutes maximum), incrémente le compteur de signature anti-rejeu, et n'autorise l'authentification qu'une seule fois par challenge.
- États de compte staff empêchant la connexion : `invited` (invitation non activée), `locked`, `disabled`.
- Le token de session staff porte les méthodes d'authentification [`'pwd'` , `'webauthn'`] — un token qui ne les contient pas exactement toutes les deux est rejeté.

1.9 Activation d'une invitation staff

- Nécessite : token d'invitation valide, mot de passe respectant la politique, et une réponse d'enregistrement WebAuthn valide (vérification utilisateur obligatoire).
- Une même identité WebAuthn (credential) ne peut être enrôlée deux fois (conflit 409).
- Succès — compte staff `active`, MFA enrôlé.

1.10 Bootstrap du premier SuperAdmin

- Commande d'amorçage unique, utilisée uniquement quand **aucun SuperAdmin actif n'existe encore**.
- Email et nom d'utilisateur validés (email : format + 255 caractères max ; nom d'utilisateur : entre 3 et 64 caractères).
- Rejeté si : un SuperAdmin actif existe déjà, si le bootstrap a déjà eu lieu, si l'email/nom d'utilisateur est déjà pris, ou si le rôle SuperAdmin n'existe pas en base (seed non appliqué).

- Une invitation avec activation MFA obligatoire est envoyée, valable **30 minutes** par défaut.

1.11 Déconnexion

- La déconnexion révoque la session correspondant au royaume (app ou admin) ; un jeton absent ou déjà invalide ne provoque pas d'erreur (opération idempotente).
-

2. Autorisations (RBAC)

2.1 Modèle

- Un compte **staff** porte un ensemble de **rôles**, chaque rôle donnant un ensemble de **permissions** explicites (pas d'héritage implicite, pas de wildcard — même le rôle SuperAdmin liste explicitement chacune de ses permissions dans les données d'amorçage).
- Toute route d'administration est protégée par une **politique d'autorisation explicite** associant méthode + chemin → permission requise. Une route sans politique déclarée est refusée par défaut (AUTH_POLICY_REQUIRED), et une politique référençant une permission inconnue est également refusée (AUTH_PERMISSION_UNKNOWN). Il n'existe pas de « laisser-passer implicite ».
- Seul un compte staff **actif** peut porter des permissions ; un compte community, même via un jeton valide, n'a jamais de permissions.

2.2 Rôles prédéfinis (données d'amorçage)

Rôle	Portée
RecipeModerator	Consultation, publication, rejet et archivage des recettes (pas de suppression définitive).
CommentModerator	Consultation, masquage, restauration et modification des commentaires (pas de suppression définitive).
UserAdmin	Consultation, bannissement et réactivation des comptes community uniquement.
CatalogManager	Gestion complète du catalogue : catégories, ingrédients (+ alias), tags, ustensiles.
SuperAdmin	Cumul explicite de toutes les permissions administratives, y compris la gestion du staff lui-même et l'audit.

2.3 Permissions notables

Les permissions sont segmentées par domaine (user.*, recipe.*, comment.*, catalog.*, staff.*, tag.*, ingredient.*, audit.read, system.health.read). Chaque action de service vérifie une permission précise (ex. : recipe.publish ≠ recipe.reject ≠ recipe.archive ≠ recipes.delete), permettant des rôles très granulaires.

2.4 Réauthentification récente exigée (« step-up »)

Certaines actions sensibles exigent une preuve d'authentification WebAuthn **récente** (par défaut : moins de 5 minutes, bornée à 10 minutes maximum), même si la session admin est encore valide :

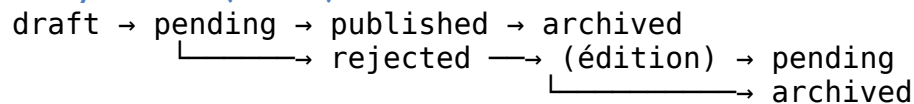
- Créer une invitation staff.
- Désactiver un compte staff.
- Octroyer **ou** révoquer spécifiquement le rôle **SuperAdmin** (les autres rôles n'exigent pas ce contrôle renforcé).

2.5 Comptes communautaires vs. staff dans les middlewares

- `CommunityOnly` : exige un compte community actif.
 - `StaffOnly` : exige un compte staff actif.
 - `RequirePermission(x)` : exige un compte staff actif portant la permission x.
-

3. Recettes

3.1 Cycle de vie (statuts)



- **draft** : recette en cours de rédaction par son auteur, non publique, entièrement éditable.
- **pending** : soumise, en attente de modération. Non éditable par l'auteur.
- **published** : validée par un modérateur, visible publiquement.
- **rejected** : refusée par un modérateur avec une raison obligatoire ; redevient éditable par l'auteur, qui peut la resoumettre.
- **archived** : retirée définitivement de la circulation (par l'auteur ou un modérateur).

3.2 Qui peut voir / éditer une recette

Action	Règle
Voir	Le staff avec la permission <code>recipe.review</code> , ou l'auteur, ou n'importe qui si la recette est <code>published</code> .
Éditer (brouillon)	Réservé à l'auteur, uniquement si le statut est <code>draft</code> ou <code>rejected</code> .
Soumettre (<code>draft/rejected</code> → <code>pending</code>)	Réservé à l'auteur d'une recette éditable ; un nouveau slug public est généré à la soumission.
Archiver par l'auteur	Réservé à l'auteur, uniquement si le statut est <code>published</code> ou <code>rejected</code> .
Modérer (approuver/rejeter)	Réservé au staff, uniquement si le statut est

Action	Règle
	pending.
Archiver par le staff	Réservé au staff, mêmes conditions de statut que l'archivage auteur (published/rejected), avec raison obligatoire.
Supprimer définitivement	Réservé au staff titulaire de la permission dédiée (recipes.delete) ; supprime aussi l'image de couverture associée en stockage.

3.3 Contenu d'une recette

- **Titre** : obligatoire, **minimum 5 caractères**.
- **Description, temps de préparation/repos/cuisson, portions** : optionnels à la saisie, valeurs par défaut appliquées à la création (description vide, 0 min de préparation, 1 portion, temps de repos/cuisson nuls).
- **Ingrédients** :
 - displayText obligatoire, non vide, **255 caractères maximum**.
 - Peut référencer un ingrédient canonique du catalogue (ingredientId) **ou** rester libre (texte non catalogué).
 - Si non catalogué, le texte est normalisé (minuscules, translittération des caractères spéciaux type œ→oe, ß→ss...) et doit produire un nom normalisé non vide d'au plus 255 caractères — sinon rejeté (RECIPES_BAD_INGREDIENT_NAME).
 - quantity (nombre ou nul), unit (texte libre ou nul), note (texte libre ou nul), sortOrder (ordre d'affichage).
- **Étapes** : chacune a un numéro (stepNumber) et une description obligatoire.
- **Ustensiles** : référence obligatoire à un ustensile du catalogue (equipmentId).
- **Tags** : liste d'identifiants de tags catalogués ; les doublons sont automatiquement dédupliqués.
- **Catégorie** : optionnelle, doit référencer une catégorie existante.

3.4 Slugs

- À la création (brouillon) : un slug technique non public est généré (draft_{userId}_{timestamp}_{aléatoire}).
- À la soumission (passage en pending) : un slug public « propre » est dérivé du titre (translittéré, minuscules, tirets), rendu unique en ajoutant un suffixe numérique (-2, -3...) en cas de collision.

3.5 Recherche et filtres publics

- Recherche plein texte (q), filtre par catégorie, par tags inclus/exclus, par ingrédients inclus/exclus, par temps total maximum.
- **Un même identifiant ne peut pas être à la fois inclus et exclu** (ex. : un tag ne peut pas figurer simultanément dans tagIds et excludedTagIds) — rejeté en cas de conflit.

- Pagination des résultats plafonnée à **50 éléments par page** (limite générale de pagination) ; le fil d'actualité (« recettes récentes ») est plafonné à **20 éléments**, 12 par défaut.

3.6 Image de couverture

Voir [section 11](#).

4. Catalogue (ingrédients, tags, catégories, ustensiles)

Le catalogue est le référentiel canonique partagé par toutes les recettes. Il est en lecture publique, mais sa gestion (création/modification/dépréciation/fusion) est réservée au staff titulaire de la permission `catalog.manage / tag.* / ingredient.*` selon le domaine.

4.1 Normalisation des noms

Pour les tags et les ingrédients, un nom canonique et son **nom normalisé** sont dérivés systématiquement : minuscules, suppression des accents/diacritiques, translittération de caractères spéciaux (æ→ae, œ→oe, ß→ss, ø→o, ð/đ→d, ł→l), puis réduction à [a-z0-9] séparés par un seul espace. Le nom normalisé fait foi pour l'unicité (deux entrées ne peuvent pas coexister avec le même nom normalisé à l'état `active`).

4.2 Cycle de vie : `active` → `deprecated` / `merged`

- **active** : entrée utilisable, visible et référençable par les recettes.
- **deprecated** : retirée de la circulation mais conservée pour historique ; ne peut être re-proposée que via restauration.
 - Un ingrédient ne peut être déprécié que s'il n'est **ni** une cible de fusion (aucun autre ingrédient fusionné vers lui), **ni** porteur d'alias actifs (les alias doivent être supprimés au préalable).
 - Un tag ne peut être déprécié que s'il n'est pas lui-même une cible de fusion.
- **merged** : fusionnée définitivement dans une autre entrée canonique active ; ne peut plus être modifiée ni re-fusionnée.
- **Restauration** : uniquement depuis `deprecated` vers `active`, et seulement si le nom normalisé n'est pas déjà repris par une autre entrée active entretemps.

4.3 Fusion (merge)

- Une entrée ne peut pas être fusionnée dans elle-même.
- La **source** ne doit pas être déjà `merged` ; la **cible** doit être `active`.
- Pour un ingrédient : toutes les associations aux recettes de la source sont transférées vers la cible (déduplication automatique), ainsi que ses alias ; le nom de la source est conservé sous forme d'**alias français préservé** sur la cible (garantissant que rien ne « disparaît » d'une recherche).
- Pour un tag : les associations aux recettes sont transférées et déduplicuées vers la cible.

- Un alias qui préserve le nom d'une source fusionnée est **protégé** : il ne peut plus être modifié ni supprimé (ADMIN_INGREDIENT_ALIASES_MERGE_SOURCE_NAME_PROTECTED).

4.4 Alias d'ingrédients

- Chaque alias est rattaché à une langue (code ISO type fr, en-US...) et n'a de sens que pour un ingrédient à l'état `active`.
- Un alias (nom normalisé + langue) doit être unique.
- Gestion réservée à la permission `ingredient.alias.manage`.

4.5 Tags & groupes

- Chaque tag appartient obligatoirement à un **groupe de tags** existant (ex. régime, saison, type de plat...).
- Slug et description optionnels, slug limité à des minuscules/chiffres/tirets simples.

4.6 Catégories & ustensiles

- Gestion simple en lecture (pas de workflow de dépréciation/fusion identifié pour ces deux entités dans la version actuelle) ; nom et slug uniques.

4.7 Règles communes de validation

- Nom : requis, non vide, 255 caractères max.
 - Slug : minuscules, chiffres et tirets simples uniquement (`^[a-z0-9]+(-[a-z0-9]+)*$`), 255 caractères max, doit rester unique.
 - Description (tags) : 1000 caractères max si fournie.
 - Code langue : `^[a-z]{2,8}(-[a-z0-9]{1,8})*$`, 35 caractères max.
 - Toute action de modération sur le catalogue (dépréciation, restauration, fusion) exige une **raison textuelle de 10 à 1000 caractères**.
-

5. Propositions de catalogue (contributions communautaires)

Un utilisateur `community` peut proposer, depuis une recette qu'il a écrite, la création d'un tag ou d'un ingrédient qui n'existe pas encore dans le catalogue.

- La recette référencée doit exister **et** appartenir à l'auteur de la proposition.
- Le nom proposé (255 caractères max) est normalisé selon les mêmes règles que le catalogue ; s'il ne produit aucun nom normalisé exploitable, la proposition est rejetée.
- Si une entrée **active** du catalogue porte déjà ce nom normalisé, la proposition est refusée (conflit) — inutile de proposer ce qui existe déjà.
- Une proposition équivalente déjà **en attente** pour la même recette ne peut pas être dupliquée.
- Chaque proposition suit un cycle de vie strict : `pending` → `accepted` / `rejected` / `merged`, avec verrouillage total après revue (immutabilité en base une fois le statut changé).
- Limite de fréquence : **10 propositions par heure** par utilisateur (tag + ingrédient confondus, cf. [12.1](#)).

Traitement par le staff (catalog.manage)

Décision	Effet
Accepter (tag)	Crée un nouveau tag actif dans le groupe indiqué, avec le nom proposé ; la proposition passe à accepted .
Accepter (ingrédient)	Crée un nouvel ingrédient actif ; la proposition passe à accepted .
Rejeter	Nécessite une raison de 10 à 1000 caractères ; aucune entrée créée.
Associer (associate)	Rattache la proposition à un tag/ingrédient existant actif déjà présent (au lieu d'en créer un nouveau) ; passe à merged .
Convertir en alias	(Ingrédients uniquement) Crée un alias linguistique sur un ingrédient cible actif existant ; passe à merged .

Toute action de revue exige une **raison de 10 à 1000 caractères**, et toute création qui entrerait en collision avec un nom déjà actif est refusée.

6. Commentaires & notation

- Un commentaire est rattaché à une recette et peut être une **réponse** à un commentaire existant, mais **un seul niveau d'imbrication est autorisé** : on ne peut pas répondre à une réponse (COMMENTS_CREATE_NESTED_REPLY).
 - **Note (rating)** : optionnelle, entier entre **1 et 5**.
 - **Une réponse ne peut pas porter de note** — la note n'a de sens que sur un commentaire racine.
 - Modification et suppression d'un commentaire réservées à son auteur (sinon 403).
 - La suppression par l'auteur est un **soft delete** (le commentaire est marqué supprimé, pas physiquement retiré).
 - **Modération staff** (comment.hide / comment.restore / comments.update / comments.delete) :
 - Masquer un commentaire (« hide ») exige une raison de 10 à 1000 caractères.
 - Un commentaire ne peut être « démodéré » ou restauré que par le staff.
 - La suppression définitive (comments.delete) est distincte du masquage et de la suppression douce par l'auteur, et est réservée à une permission spécifique.
-

7. Favoris

- Un favori associe un utilisateur community à une recette (une seule fois : clé primaire composite utilisateur + recette).

- Ajout et retrait libres pour l'utilisateur connecté ; aucune règle métier supplémentaire (pas de restriction sur les recettes non publiées côté service, la visibilité est gérée en amont par le contrôleur/la recherche).
-

8. Modération & journal d'audit administratif

8.1 Principe : audit obligatoire et infaillible (« fail-closed »)

Toute action administrative sensible (modération, gestion du staff, gestion du catalogue, listing avec effet de traçabilité) s'exécute dans une **transaction unique** qui doit produire **exactement un enregistrement d'audit** :

- Si l'action réussit mais que l'audit ne peut pas être écrit → la transaction entière est annulée (rollback), et l'action est refusée avec l'erreur `ADMIN_AUDIT_RECORD_FAILED`. **Aucune action administrative ne peut aboutir sans laisser de trace.**
- Si une action produit zéro ou plus d'un enregistrement d'audit (incohérence de code), la transaction est également annulée.
- Exception explicite : une action qui renvoie `false` (ex. : rien à faire, condition non remplie) est autorisée à ne produire aucun enregistrement.

8.2 Contenu de l'audit

Chaque entrée d'audit capture : l'acteur (identifiant staff), le type d'événement (catalogue fermé de valeurs, ex. `recipes.approve`, `staff.disable`...), le type de cible et son identifiant, une raison optionnelle, un instantané *avant* et *après* (JSON), l'adresse IP, le user-agent, et un identifiant de corrélation UUID.

- Chaque type d'événement est associé de façon **rigide** à un type de cible (ex. `staff.disable` → cible obligatoirement `staff_user`) ; toute incohérence est rejetée.
- **Rédaction automatique des champs sensibles** : tout champ dont le nom contient (une fois normalisé) `password`, `secret`, `token`, `credential`, ou qui correspond exactement à `apikey/authorization/cookie/privatekey`, voit sa valeur remplacée par `[REDACTED]` avant stockage — y compris récursivement dans les objets et tableaux imbriqués.
- Les instantanés doivent être des objets JSON « plats » valides (pas de références circulaires, uniquement des valeurs JSON).

8.3 Journaux d'audit immuables

- Les journaux de modération (recettes, commentaires) et les propositions de catalogue déjà revues sont **en lecture seule après écriture** : toute tentative de modification ou de suppression déclenche une erreur SQL explicite (« append-only »).
- Un enregistrement de modération de recette/commentaire conserve l'identifiant métier même après une suppression définitive autorisée (pas de clé étrangère vers l'entité, volontairement, pour préserver l'historique).

8.4 Raisons de modération : règle générale

Quasi toutes les actions administratives destructives ou correctives (bannir/débannir un utilisateur, rejeter/archiver une recette, masquer un commentaire, désactiver/activer un staff, accorder/révoquer un rôle, révoquer une session staff gérée, réviser une proposition de catalogue, déprécier/restaurer/fusionner un élément du catalogue) exigent une **raison textuelle non vide de 10 à 1000 caractères**. C'est une règle transversale volontairement uniforme dans tout le système.

9. Gestion du personnel (Staff)

9.1 Invitations staff

- Réservée à la permission `staff.create`, avec **réauthentification récente obligatoire**.
- Champs : email (format valide, 255 caractères max), nom affiché (3 à 64 caractères), liste de rôles initiaux (1 à 20 rôles, codes uniques, 64 caractères max chacun).
- Refusée si : une invitation existe déjà pour cet email, l'email ou le nom affiché sont déjà utilisés, ou un des rôles indiqués n'existe pas.
- Invitation valable **24 heures** par défaut (1440 minutes), à usage unique.
- L'email d'invitation impose que l'activation MFA est obligatoire.

9.2 Rôles staff

- Octroi/révocation réservés respectivement à `staff.role.grant` / `staff.role.revoke`.
- **Un membre du staff ne peut ni s'octroyer ni se révoquer un rôle à lui-même** (`STAFF_ROLE_GRANT_SELF_FORBIDDEN` / `STAFF_ROLE_REVOKE_SELF_FORBIDDEN`).
- On ne peut pas accorder un rôle déjà détenu, ni révoquer un rôle non détenu (conflit).
- **Protection du dernier SuperAdmin actif** : il est impossible de révoquer le rôle SuperAdmin du dernier compte actif qui le détient (`LAST_ACTIVE_SUPER_ADMIN`) — condition vérifiée avec verrouillage pour éviter toute concurrence.
- Octroi ou révocation du rôle **SuperAdmin** spécifiquement exige une réauthentification récente (voir 2.4).

9.3 Cycle de vie d'un compte staff

- **Désactivation** (`staff.disable`) :
 - Un staff ne peut pas se désactiver lui-même.
 - Impossible si c'est le dernier SuperAdmin actif.
 - Seul un compte `active` peut être désactivé.
 - Toutes les sessions actives du compte sont révoquées en même temps.
 - Réauthentification récente obligatoire.
- **Réactivation** (`staff.enable`) :
 - Seul un compte `disabled` peut être réactivé.

- Le MFA doit déjà avoir été enrôlé (sinon impossible de réactiver — cohérent avec l’obligation MFA pour tout compte active).

9.4 Sessions staff

- Un staff peut lister et révoquer ses **propres** sessions actives à tout moment.
 - Un staff avec la permission `staff.session.revoke` peut lister/révoquer les sessions d’un **autre** compte staff, motif obligatoire de 10 à 1000 caractères, typé comme révocation pour « compromission suspectée ».
 - Une session non trouvée/déjà inactive renvoie une erreur 404 explicite plutôt qu’un succès silencieux.
-

10. Formulaire de contact

- Champs requis : nom, email, sujet, message.
 - Contraintes de longueur : nom 2–100, email ≤ 255 (+ format valide), sujet 3–150, message 10–5000 caractères.
 - Limite de fréquence : **5 messages par heure** par adresse IP (voir 12.1).
 - L’échec d’envoi SMTP renvoie une erreur dédiée (`CONTACT_SEND_FAILED`) sans exposer les détails internes.
-

11. Images de recettes

11.1 Contraintes de fichier

- Une recette n’a **qu’une seule** image de couverture à la fois (remplacement, pas d’accumulation).
- Poids maximum : **10 Mo**.
- Dimension maximale (largeur ou hauteur) : **10 000 pixels**, avec une limite de décodage des pixels alignée dessus (protection contre les « bombes décompressives »).
- Formats acceptés après décodage réel du contenu (pas seulement l’extension) : **JPEG, PNG, WebP**. Toute signature binaire reconnue comme un autre format (GIF, BMP, TIFF, PDF, SVG, conteneurs ISO media type MP4...) est explicitement rejetée, de même qu’un contenu dont le format ne peut être déterminé.
- L’orientation EXIF est prise en compte pour déterminer les dimensions « logiques » réelles (rotation 90°/270° inversant largeur/hauteur).

11.2 Traitement

- Trois variantes sont systématiquement générées en WebP (qualité 82) : **large** (1600px), **medium** (800px), **thumbnail** (400px), redimensionnées par contrainte (jamais agrandies au-delà de l’original).
- Le texte alternatif (`alt`) est optionnel, 255 caractères max, ne doit contenir aucune balise HTML.

11.3 Autorisation

- Remplacer ou supprimer l'image d'une recette suit exactement la même règle que l'édition du contenu de la recette (auteur, recette en `draft` ou `rejected`).
 - En cas d'échec après téléversement partiel vers le stockage (S3/local), les objets déjà envoyés sont nettoyés pour éviter les orphelins ; si le remplacement réussit, les anciens fichiers sont supprimés après coup (jamais avant, pour éviter une fenêtre sans image).
 - La suppression définitive d'une recette par le staff déclenche également le nettoyage de ses images en stockage.
-

12. Sécurité transverse

12.1 Limitation de fréquence (rate limiting)

Domaine	Limite par défaut
Authentification (inscription, connexion, validation email, mot de passe oublié, connexion staff, enrôlement MFA, activation d'invitation)	5 tentatives / 15 minutes par clé (IP + méthode + route)
Formulaire de contact	5 messages / heure
Propositions de catalogue (tag + ingrédient)	10 propositions / heure

Chaque réponse inclut les en-têtes `RateLimit-Limit`, `RateLimit-Remaining`, `RateLimit-Reset`, et `Retry-After` en cas de dépassement (429).

12.2 Sessions, cookies et JWT

- Deux royaumes strictement séparés : **app** (community) et **admin** (staff), avec noms de cookies, audiences JWT et durées de vie **obligatoirement différentes** — la configuration refuse de démarrer si :
 - les deux audiences JWT sont identiques,
 - les deux noms de cookies sont identiques,
 - la durée de vie du JWT admin n'est pas strictement plus courte que celle de l'app,
 - la durée de vie du cookie admin n'est pas strictement plus courte que celle de l'app.
- Durées par défaut : session app **7 jours**, session admin **8 heures**.
- Un jeton de session app doit porter exactement la méthode d'authentification `[ 'pwd' ]` ; un jeton admin doit porter exactement `[ 'pwd' , 'webauthn' ]`. Tout écart invalide le jeton.
- Le coût de hachage `bcrypt` par défaut est **12**.
- L'origine `WebAuthn` doit être en HTTPS en dehors de `localhost/127.0.0.1`, et l'identifiant de « relying party » (RP ID) doit correspondre au nom d'hôte de cette origine (ou à un de ses domaines parents).

12.3 CORS

- Liste blanche explicite d'origines (CORS_ALLOWED_ORIGINS), avec identifiants de connexion (cookies) autorisés.
 - Le caractère générique * est explicitement interdit dès lors que les identifiants sont autorisés (le serveur refuse de démarrer sinon).
-

13. Annexe — invariants au niveau base de données

Ces règles sont appliquées directement par le schéma MySQL (contraintes CHECK, déclencheurs), en plus (ou en complément défensif) des vérifications applicatives ci-dessus :

- **Recettes** : une raison de rejet est obligatoire et bornée (10–1000 caractères) si et seulement si le statut est `rejected` ; une raison d'archivage, si fournie, est bornée de la même façon.
 - **Ingrédients / Tags** : le nom normalisé stocké doit correspondre exactement au résultat déterministe de la normalisation du nom affiché (garantie de cohérence, pas seulement une convention applicative) ; une entrée `merged` doit obligatoirement pointer vers une entrée cible, et une entrée `active/deprecated` ne doit jamais en avoir ; un ingrédient/tag référencé par au moins une recette ne peut pas être fusionné tant que ces associations existent encore ; les tags ne peuvent **jamais** être supprimés physiquement (seulement dépréciés ou fusionnés) — une tentative de `DELETE` échoue systématiquement.
 - **Alias d'ingrédients** : ne peuvent référencer qu'un ingrédient canonique `active` ; un alias qui préserve le nom d'origine d'une fusion ne peut être ni réassigné ni supprimé.
 - **Propositions de catalogue** : doivent être créées au statut `pending` ; une fois revues, l'identité de la proposition (auteur, recette, type, nom) devient immuable, tout comme son statut ; une proposition acceptée/fusionnée doit référencer une entrée catalogue `active` correspondant exactement à son type ; suppression physique interdite (historique permanent).
 - **Commentaires** : une note, si présente, doit être comprise entre 1 et 5 ; les champs de modération (date, auteur, raison) doivent être renseignés tous ensemble ou tous absents (jamais un état partiel).
 - **Comptes** : un compte `community` doit toujours avoir un mot de passe ; un compte `staff` actif doit toujours avoir un mot de passe et un MFA enrôlé ; les journaux de modération (recettes, commentaires, staff) sont en écriture seule après création (`UPDATE/DELETE` bloqués par déclencheur).
 - **Staff** : un staff ne peut pas se cibler lui-même dans une demande de changement de privilège ; le numéro de version de session (anti-rejeu WebAuthn) doit toujours être strictement positif.
-

Document de référence à faire évoluer avec le code. En cas de divergence, le code source (services, DTO, migrations SQL) fait foi.